

FORMULACIÓN DEL PROYECTO DE TÍTULO

Diseño y Desarrollo de una Plataforma Cloud (SaaS) para la Gestión, Arriendo Flexible y Legalización de Espacios Comerciales e Inmobiliarios con Enfoque en Privacidad desde el Diseño

Asignatura: Proyecto de Título - TIH184

Académico guía: Teresa Jesús Tapia Soto

Integrantes del equipo: Hernán Espinoza, Anita Marchant, Erick Silva

Fecha de entrega: 15 de septiembre del 2026

I. Introducción

El presente informe documenta la formulación de la Evaluación Sumativa 1 (ES1) del Proyecto de Título, enfocado en resolver la ineficiencia, fricción legal y vulnerabilidad de datos en el mercado de arriendo de espacios ociosos en Chile. Tras los cambios en las dinámicas laborales, existe una alta desocupación de infraestructura comercial. Para abordar esto, el equipo propone el desarrollo de un *Marketplace* y software de gestión (SaaS) que permite rentabilizar estos espacios de forma flexible.

El proyecto destaca por su integración tecnológica de vanguardia, operando 100% online mediante arquitectura *Cloud-Native* basada en contenedores (Docker y Google Cloud). El sistema orquestrará tres ejes críticos: la validación financiera mediante *Split Payments* y retención (Escrow); la legalización remota mediante integración con notarías virtuales para el estricto cumplimiento de la "Ley Devuélveme mi Casa" (Ley 21.461); y un ecosistema de registro único que garantiza el almacenamiento seguro de contratos y documentos sensibles. Esto da estricto cumplimiento a la nueva Ley sobre Protección de Datos Personales (Ley 21.719) bajo el estándar de *Privacy by Design*, implementando trazabilidad de eventos mediante arquitecturas de datos duales (OLTP y OLAP).

II. Identificación del Problema

2.1 Actualización y justificación del problema

2.1.1 Descripción de la organización.

2.1.1.1 Antecedentes de la Organización.

El proyecto se plantea bajo la modalidad de **Emprendimiento** (Startup Tecnológica). Se trata de la creación de una nueva empresa proveedora de servicios de software orientada al rubro *prop-tech* y *legal-tech*.

A través del modelo CANVAS, la organización se define con:

- **Segmento de Clientes:** B2B (Empresas/Pymes con bodegas u oficinas ociosas) y B2C (Usuarios que buscan arrendar espacios eventuales o a largo plazo).

- **Propuesta de Valor:** Flexibilización del arriendo de espacios eliminando la burocracia presencial, garantizando seguridad jurídica (contratos notariales online), seguridad financiera (Escrow) y privacidad absoluta de los documentos sensibles del usuario.
- **Estructura de Ingresos:** Cobro de comisión por transacción (*Take Rate*) y traspaso de costos de API por gestión legal de documentos.

2.1.1.2 Diagnóstico de la situación actual.

Actualmente, el proceso de arriendo de espacios es rígido y presencial. Los dueños temen arrendar por la inseguridad jurídica frente a morosidad. Además, la gestión documental actual es altamente insegura: dueños y arrendatarios comparten cédulas de identidad, carpetas tributarias y contratos firmados a través de medios no cifrados como WhatsApp o correos electrónicos personales. Esto los expone a graves vulneraciones de privacidad, robo de identidad y contraviene explícitamente los mandatos de la Ley 21.719.

2.1.2 Descripción del problema.

El problema central es la **desocupación ineficiente de infraestructura comercial y la alta fricción legal, sumada a la vulnerabilidad en el tratamiento de datos personales y financieros de las partes involucradas**. No existe una plataforma unificada en Chile que resuelva simultáneamente el catálogo de espacios, la validación tributaria (SII), el pago condicionado, la firma de contratos notariales y el resguardo seguro y auditable de dichos documentos.

2.2 Justificación del problema

2.2.1 Relevancia del problema.

Resolver este problema mitiga las pérdidas económicas de los propietarios y facilita el acceso a pymes. Adicionalmente, al centralizar la identidad y los documentos en una bóveda *Cloud* autenticada, se protege a los usuarios del fraude y se profesionaliza un rubro que actualmente opera en la informalidad digital, asegurando el cumplimiento legal (Ley 21.461 y Ley 21.719) desde el diseño inicial de la plataforma.

2.2.2 Complejidad del problema.

La complejidad radica en la arquitectura de software distribuida. Implica crear un registro único centralizado (Autenticación JWT en Go), orquestar múltiples APIs (FirmaVirtual para ANF y Mercado Pago para Escrow), mantener la consistencia transaccional financiera (ACID en PostgreSQL), gestionar un almacenamiento de objetos seguro (*Google Cloud Storage*) para las plantillas de contratos y separar la carga de auditoría legal hacia un almacén de datos (Google BigQuery). Todo esto dentro de contenedores orquestados de manera remota e integrados mediante flujos de CI/CD.

III. Levantamiento de Requerimientos

3.1 Determinación de los instrumentos a utilizar.

Se utilizó una metodología adaptativa (Ágil), documentando bajo el formato de **Historias de Usuario (User Stories)**. Se realizó observación del mercado inmobiliario, revisión técnica de documentación de APIs (Mercado Pago, FirmaVirtual), análisis de arquitecturas Cloud y revisión del marco regulatorio actual (Leyes 21.461 y 21.719).

3.2 Detalle de los requerimientos.

- **HU01 - Contratos Dinámicos:** Como arrendador, quiero que el sistema genere automáticamente el contrato y exija Autorización Notarial de Firma (ANF) o Firma Electrónica Avanzada (FEA) dependiendo del plazo del arriendo, para estar protegido bajo la ley "Devuélveme mi casa".
- **HU02 - Escrow (Retención de Pagos):** Como cliente, quiero que mi dinero quede retenido en la plataforma y solo se libere al arrendador cuando se cumplan las condiciones del arriendo.
- **HU03 - Validación Tributaria (KYB):** Como plataforma, quiero validar que el arrendador posea un giro compatible en el SII (ej. 681011 o 681012) antes de permitirle publicar espacios comerciales.
- **HU04 - Registro Único y Centralizado:** Como usuario, quiero registrarme una sola vez en el sistema para poder actuar tanto como arrendador o arrendatario, manteniendo un único perfil seguro.
- **HU05 - Almacenamiento Cifrado de Contratos:** Como sistema, requiero almacenar las cédulas de identidad y los contratos PDF en un *Bucket* seguro (*Google Cloud Storage*), restringiendo el acceso solo mediante URLs firmadas temporalmente.
- **HU06 - Derecho al Olvido y Auditoría:** Como usuario, quiero poder solicitar la eliminación de mi cuenta personal. Como administrador, requiero que toda acción en el sistema genere un log inmutable y anonimizado en BigQuery para cumplir con auditorías legales (derechos ARCOP).

IV. Marco Teórico

4.1 Investigación Bibliográfica.

- **Ley de Arriendos (Ley 21.461 - "Devuélveme mi casa"):** Exige contratos con firmas autorizadas ante notario para agilizar juicios de desalojo monitorio, justificando la integración de una API de notaría virtual.
- **Nueva Ley de Protección de Datos Personales (Ley 21.719):** Impone los principios de licitud, finalidad y *Privacy by Design*. Fundamenta el uso de almacenamiento centralizado, autenticación segura y registros (logs) auditables.
- **Arquitecturas Cloud y Contenedores (Docker):** El uso de contenedores aísla los procesos y asegura que los entornos de desarrollo local y producción (*Google Cloud Run*) sean idénticos, erradicando problemas de dependencias.
- **Sistemas OLTP vs OLAP:** Diferenciación en el almacenamiento de datos. Las bases transaccionales (OLTP como PostgreSQL) garantizan propiedades ACID para dinero y contratos. Los almacenes de datos (OLAP como BigQuery) permiten el procesamiento

analítico y el resguardo histórico de logs de manera escalable.

- **Manejo de Concurrencia (Golang):** Lenguaje que proporciona *Goroutines*, permitiendo que el backend maneje eficientemente múltiples hilos de ejecución simultáneos (ej: procesar un pago y enviar un log asíncrono a BigQuery) con bajo consumo de recursos.

V. Objetivos del Proyecto

5.1 Solución tecnológica

5.1.1 Formulación de la Solución.

Se desarrollará una **Plataforma Web SaaS basada en contenedores**. El Frontend será construido en Next.js. El Backend (Golang) actuará como orquestador central gestionando la autenticación (JWT) y las reglas de negocio. La persistencia operativa residirá en PostgreSQL, mientras que los archivos irán a Cloud Storage y la auditoría a BigQuery. Todos los servicios vivirán en contenedores (Docker) alojados en Google Cloud Platform, integrándose con Mercado Pago (Split Payments) y FirmaVirtual (Notaría API).

5.1.2 Alcance y restricciones.

- **Alcance:** Pasarela de pagos con retención, generación de contratos, firma notarial remota, almacenamiento seguro de documentos, flujos de auditoría (logs) y automatización CI/CD.
- **Restricciones:** El proyecto es un intermediario tecnológico. No asume responsabilidad penal sobre daños a la propiedad ni actúa como administrador físico de recintos.

5.2 Impacto de la solución

5.2.1 Proceso de negocio afectado.

Optimiza el "**Acuerdo Comercial Inmobiliario**" y la "**Gestión Documental**", reduciendo tiempos de validación y bloqueando la fuga de datos personales al formalizar el rubro.

5.2.2 Registro de Interesados.

1. **Arrendadores y Arrendatarios:** Buscan transacciones seguras jurídicamente y resguardo de datos.
2. **Proveedores API:** Mercado Pago (FinTech), FirmaVirtual (LegalTech).

5.2.3 Indicadores de gestión.

- Tiempo de ciclo de firma de contrato.
- Tasa de cumplimiento de privacidad (Solicitudes procesadas, URLs firmadas generadas).

5.2.4 Niveles de servicio.

Disponibilidad mediante infraestructura Serverless (*Cloud Run*). Trazabilidad de logs de auditoría en tiempo real.

5.3 Objetivos del proyecto

5.3.1 Objetivo General.

Desarrollar e implementar una plataforma web transaccional basada en arquitectura Cloud para la gestión y arriendo flexible de espacios, automatizando la firma notarial y retención de pagos, y asegurando el estricto cumplimiento de la Ley 21.719 mediante el resguardo cifrado de datos e identidades bajo un modelo auditable de bases de datos operativas y analíticas.

5.3.2 Objetivos Específicos.

1. **Diseñar e implementar** la infraestructura en contenedores (Docker) para su despliegue en Google Cloud Platform utilizando integración continua (CI/CD con GitHub).
2. **Desarrollar** un backend concurrente en Golang que centralice la autenticación y gestione el almacenamiento en *Cloud Storage*.
3. **Integrar** los servicios de validación y firma notarial consumiendo la API de FirmaVirtual de forma dinámica según el tipo de arriendo.
4. **Implementar** el módulo transaccional (Mercado Pago) acoplado a PostgreSQL para flujos *Escrow* atómicos.
5. **Configurar** un módulo de auditoría asíncrono utilizando Google BigQuery para la trazabilidad inmutable de eventos, dando cumplimiento a normativas de protección de datos.

VI. Metodología de Trabajo

6.1 Metodología de Desarrollo.

Enfoque **Ágil (Scrum)** apoyado por cultura **DevOps**. Se usará GitHub y flujos basados en ramas (*Git Flow*) permitiendo desarrollo en paralelo. Se utilizará docker-compose para pruebas locales, garantizando entregas continuas en ciclos cortos y revisiones periódicas iterativas.

6.2 Duración y cronograma.

Ciclo de 3 a 4 meses (12-16 semanas):

- **Semanas 1-4:** Levantamiento, diseño relacional (PostgreSQL), configuración de contenedores (Docker) y repositorios GitHub. Maquetación (Next.js).
- **Semanas 5-8:** Desarrollo de autenticación y flujos de gestión de archivos (Cloud Storage) y logs de auditoría (BigQuery) en Golang.
- **Semanas 9-12:** Integraciones APIs (Mercado Pago, FirmaVirtual) y lógica de validación SII.
- **Semanas 13-16:** CI/CD hacia GCP (*Cloud Run*), pruebas de carga, revisión de cumplimiento de Ley 21.719 y documentación final técnica.

6.3 Equipo de trabajo (Matriz RACI base).

- **Hernán Espinoza:** Arquitecto Cloud y Backend. Encargado de infraestructura

(Docker/GCP), CI/CD (GitHub Actions), Base de Datos (PostgreSQL/BigQuery) y estrategias de auditoría de datos.

- **Erick Silva:** Especialista en Integraciones y Reglas de Negocio. Encargado del consumo de APIs (Mercado Pago, FirmaVirtual) y definición de la lógica de validación tributaria.
- **Anita Marchant:** Desarrolladora Frontend. Encargada del desarrollo en Next.js, interfaces de usuario (UI/UX), dashboards de gestión de arriendos y consumo de la API REST interna.

6.4 Plan de recursos.

- **Hardware/Software:** 3 computadores corriendo Docker Desktop. GitHub para control de versiones.
- **Infraestructura Cloud:** Google Cloud Platform (Cloud Run, Cloud SQL/Compute Engine, Cloud Storage, BigQuery), utilizando créditos de prueba (\$300 USD) para mantener el costo operativo en \$0 durante el desarrollo.

VII. Definición y Justificación de Arquitectura TI (ADR)

7.1 Tipo de arquitectura TI y compatibilidad.

Se define una **Arquitectura Cloud-Native en Contenedores orientada a Microservicios**. Se implementa un patrón de bases de datos bimodales (OLTP para operación y OLAP para auditoría).

7.2 Matrices de Decisión Arquitectónica (ADR) y Descarte Tecnológico

Para justificar las elecciones tecnológicas y asegurar que la plataforma pueda desarrollarse eficientemente por un equipo distribuido, se evaluaron diversas alternativas.

7.2.1 Desarrollo Frontend (Capa de Presentación)

Criterio Técnico	Next.js (Elegido)	React SPA (Descartado)	Angular (Descartado)
SEO y Renderizado (SSR)	Excelente (Nativo en servidor)	Pobre (Renderiza en cliente)	Requiere Angular Universal
Tiempo de Carga	Rápido (Server Components)	Lento (Carga todo el JS)	Lento (Bundle muy pesado)
Decisión Final	SELECCIONADO	DESCARTADO	DESCARTADO

- **Justificación:** React puro genera una *Single Page Application* (SPA), lo que penaliza severamente la indexación en motores de búsqueda (SEO), algo crítico para un catálogo de arriendos. Next.js soluciona esto generando las páginas desde el servidor. Angular representa una sobreingeniería innecesaria para los tiempos del proyecto.

7.2.2 Desarrollo Backend (Capa Lógica y Orquestación)

Criterio Técnico	Golang - Go (Elegido)	Node.js (Descartado)	Python (Descartado)
Concurrencia	Nativo y ligero (<i>Goroutines</i>)	<i>Single-threaded</i> (Event Loop)	Lento (Bloqueo GIL)
Rendimiento Cloud	Binario compilado (Arranca ms)	Requiere cargar V8	Requiere intérprete Python
Decisión Final	SELECCIONADO	DESCARTADO	DESCARTADO

- **Justificación:** El backend debe procesar peticiones de pagos, PDFs, notificaciones y enviar logs a BigQuery simultáneamente. Golang maneja alta concurrencia consumiendo mínimos recursos, abaratando drásticamente los costos de alojamiento en Cloud Run frente a Node.js o Python.

7.2.3 Base de Datos Operativa (Capa de Persistencia Financiera)

Criterio Técnico	PostgreSQL (Elegido)	DynamoDB / NoSQL (Descartado)
Tipo de carga	OLTP (Transaccional)	OLTP NoSQL (Clave-Valor)
Integridad / ACID	100% (Bloqueos y rollback atómico)	Limitada y costosa para relaciones
Búsqueda / Filtros	Flexible (Múltiples facetas y JOINS)	Rígida (Requiere índices GSI)
Decisión Final	SELECCIONADO	DESCARTADO

- **Justificación:** Aunque modelos NoSQL como *Single-Table Design* son rápidos para lecturas por clave, un marketplace inmobiliario requiere integridad referencial estricta (Usuarios → Espacios → Contratos → Pagos). PostgreSQL garantiza transacciones ACID, evitando que un pago se cobre si la actualización del contrato falla, lo que es vital para un sistema de retención monetaria.

7.2.4 Infraestructura de Hosting (Capa de Despliegue)

Criterio Técnico	Google Cloud Platform (Elegido)	VPS Tradicional (Descartado)
------------------	---------------------------------	------------------------------

Gestión de Servidores	Administrado (<i>Serverless/Run</i>)	Manual (Linux, Nginx, SSL)
Escalabilidad	Automática por demanda	Manual (Upgrades rígidos)
Decisión Final	SELECCIONADO	DESCARTADO

- **Justificación:** Configurar un VPS obligaría al equipo a gastar semanas en tareas operativas (firewalls, certificados HTTPS). Google Cloud Platform (GCP) permite desplegar contenedores con auto-escalado inmediato, alineándose a los créditos universitarios disponibles y elevando el nivel de ingeniería del proyecto.

7.2.5 Empaquetado y Orquestación (Capa de Ejecución)

Criterio Técnico	Docker / Contenedores (Elegido)	Instalación Nativa (Descartado)
Consistencia	Garantizada ("Funciona en todas partes")	Fricción ("En mi máquina funciona")
Agilidad	Excelente (docker-compose up)	Lenta (Instalación de dependencias)
Decisión Final	SELECCIONADO	DESCARTADO

- **Justificación:** Que cada integrante instale bases de datos y lenguajes nativamente genera graves conflictos de versiones. Docker aísla todo el ecosistema, garantizando que el entorno local de cada estudiante sea idéntico al que se ejecutará en los servidores de Google Cloud.

7.2.6 Control de Versiones y Trabajo Colaborativo (CI/CD)

Criterio Técnico	GitHub + Git Flow (Elegido)	Compartir archivos/Drive (Descartado)
Trabajo Remoto	Excelente (Ramas independientes)	Desastroso (Sobreescritura de código)
Automatización	Integrado (GitHub Actions CI/CD)	Despliegue manual (FTP)
Decisión Final	SELECCIONADO	DESCARTADO

- **Justificación:** El uso de GitHub permite trabajar en ramas separadas simultáneamente. Al fusionar el código, se ejecutan *pipelines* de GitHub Actions (CI/CD) que construyen la imagen Docker y la despliegan automáticamente en GCP, eliminando la intervención

manual en el servidor.

7.2.7 Proveedor de Firma Digital y LegalTech

Criterio Legal	FirmaVirtual API (Elegido)	Firma Simple FES / Casilla (Descartado)
Juicio Desalojo	Sí (Ofrece Autorización Notarial ANF)	Inválido para desalojo express
Protección Dueño	Total	Nula (Repudiable en tribunales)
Decisión Final	SELECCIONADO	DESCARTADO

- **Justificación:** Un simple clic de "Términos y Condiciones" (FES) no sirve para un contrato de arriendo bajo la Ley 21.461. FirmaVirtual permite enrutar contratos dinámicamente: FEA para eventos cortos y ANF (Notarial) para arriendos largos, protegiendo jurídicamente a los arrendadores.

7.2.8 Pasarela de Pagos (Flujo de Retención / Escrow)

Criterio Financiero	Mercado Pago (Elegido)	Transferencia Manual (Descartado)
Modelo Split	Sí (Nativo y disponible en Chile)	No aplica
Retención Escrow	Mediante estado de acreditación y API	Manual y riesgoso
Decisión Final	SELECCIONADO	DESCARTADO

- **Justificación:** Para operar como Marketplace y cobrar comisiones sin desórdenes tributarios, se requiere *Split Payments*. Mercado Pago permite cobrar, retener los fondos temporalmente en la plataforma y luego dispersar el dinero automáticamente cuando finalice el arriendo.

7.3 Estrategia de Auditoría y Trazabilidad Legal (Cumplimiento Ley 21.719)

Para dar cumplimiento a la nueva Ley sobre Protección de los Datos Personales (Ley 21.719), la plataforma requiere un sistema de registro de eventos (logs) que sea auditable, inmutable y seguro. Este sistema debe registrar cada inicio de sesión, creación de cuentas, modificación de contratos y borrado de datos (Derecho al Olvido), separando la carga analítica de la base de datos operativa.

7.3.1 Implementación de Logs Auditable en el Backend (Go)

El ecosistema implementará un patrón de **Middleware de Intercepción** en el backend desarrollado en Go. El flujo operativo se estructura de la siguiente manera:

1. **Captura del Evento:** Cada vez que un usuario interactúa con un endpoint crítico (ej. firmar un contrato, eliminar su cuenta, solicitar un retiro de dinero), el middleware intercepta la petición HTTP.
2. **Anonimización (Privacy by Design):** Para proteger la Información Personal Identificable (PII), los logs de acceso no guardarán correos electrónicos ni RUTs, registrando de manera exclusiva los identificadores únicos universales (UUIDs) de los usuarios. Se añadirá el *timestamp* exacto (marca de tiempo), la dirección IP hasheada y el código de la acción realizada (ej. USER_LOGIN, CONTRACT_SIGNED, DATA_DELETED).
3. **Procesamiento Asíncrono:** Utilizando las *Goroutines* nativas de Go, el registro del log se procesa en un hilo secundario de manera asíncrona. Esto garantiza que el usuario no experimente latencia mientras el sistema guarda el registro de auditoría.
4. **Ingesta en BigQuery:** El backend envía este registro directamente a Google BigQuery mediante su API de *Streaming Inserts*, creando un rastro de auditoría de solo escritura (*append-only*).

7.3.2 Arquitectura de Datos: Separación OLTP y OLAP

La plataforma adopta una arquitectura de datos bimodal para garantizar tanto el rendimiento operativo como la capacidad analítica y de auditoría:

- **Capa Operativa (OLTP - Cloud SQL / PostgreSQL):** Mantiene el estado actual del sistema. Gestiona las retenciones de dinero, el control de inventario de los espacios y la disponibilidad en tiempo real.
- **Capa Analítica y de Auditoría (OLAP - Google BigQuery):** Actúa como la bóveda inmutable de la plataforma. Almacena el historial completo de eventos del sistema (logs) para responder a requerimientos legales de la Ley 21.719 y permite procesar métricas de inteligencia de negocios (ej. tasas de ocupación histórica de bodegas, demanda por temporada) sin consumir los recursos de la base de datos principal.

7.3.3 Matriz de Decisión Arquitectónica (ADR): Almacenamiento de Logs de Auditoría

Para justificar la elección de Google BigQuery como repositorio de auditoría, se evaluaron tres alternativas arquitectónicas, considerando los requisitos legales de trazabilidad y el modelo de negocio:

Criterio Técnico	Google BigQuery (Elegido)	PostgreSQL (Tabla logs)	Cloud Storage / AWS S3
Naturaleza del Almacenamiento	Data Warehouse (OLAP). Optimizado para anexar datos.	Base de datos transaccional (OLTP).	Almacenamiento de objetos (<i>Buckets</i>).

Inmutabilidad y Seguridad	Excelente. Se configuran permisos de "solo inserción" (<i>append-only</i>), evitando manipulaciones.	Riesgo Medio. Un usuario con acceso a la base operativa podría ejecutar un UPDATE o DELETE sobre los logs.	Alta inmutabilidad, requiriendo almacenar los logs en formato JSON o CSV.
Capacidad de Consulta y Búsqueda	Sobresaliente. Permite ejecutar consultas SQL sobre petabytes de datos en segundos para auditorías legales.	Deficiente a gran escala. Las tablas de logs crecen rápidamente, degradando el rendimiento de las operaciones comerciales.	Nula por defecto. Requiere descargar los archivos o integrar herramientas externas para leer el contenido.
Impacto en el Rendimiento Web	Nulo. La carga analítica se procesa en clústeres independientes de Google.	Alto. Buscar un log histórico consume CPU y memoria de la base de datos que atiende los arriendos.	Nulo, pero añade sobrecarga de procesamiento al momento de estructurar los archivos.
Decisión Final	SELECCIONADO	DESCARTADO	DESCARTADO (Se reserva solo para PDFs y fotos).

- **Justificación de Descarte Técnico:**

- **Por qué NO usar una tabla en PostgreSQL:** Almacenar el registro histórico de cada clic y transacción en la misma base de datos que maneja las reservas activas es un antipatrón de diseño. A los pocos meses, la tabla de logs saturaría el almacenamiento y la memoria RAM de la instancia (Compute Engine / Cloud SQL), ralentizando la experiencia de los clientes que buscan arrendar espacios.
- **Por qué NO usar Cloud Storage para logs estructurados:** Aunque guardar logs en buckets de almacenamiento en la nube es económico e inmutable, carece de capacidades nativas de consulta estructurada. Si el ente regulador solicita un informe de accesos, buscar esa información en miles de archivos de texto plano dentro de un *bucket* sería logísticamente inviable. BigQuery resuelve esto ofreciendo almacenamiento masivo con un motor SQL integrado.

VIII. Reconocimiento de arquitectura empresarial

La plataforma se modela como un **Ecosistema SaaS Multisided (Marketplace)**. Actúa como el nodo central de confianza que conecta a dueños de infraestructura y clientes finales. Al integrar

el flujo legal, la validación tributaria y el ecosistema financiero, la solución moderniza a las pymes inmobiliarias, garantizando resguardo bajo la normativa (Ley 21.719) en la nube de Google.

IX. Conclusiones

El desarrollo de esta plataforma demuestra que la modernización del rubro inmobiliario requiere orquestación profunda e ingeniería de software avanzada. Al resolver simultáneamente la fricción legal (API de notarías) y la gestión atómica del dinero, se mitiga el riesgo histórico del arriendo.

Al incorporar el concepto de *Privacy by Design* separando la carga operativa (PostgreSQL) de la analítica (BigQuery), el equipo justifica sólidamente el uso de Google Cloud, Docker y Go. Esto resulta en un producto corporativo, asíncrono y académicamente impecable para la evaluación final.

X. Referencias bibliográficas

- Biblioteca del Congreso Nacional de Chile. (2022). *Ley 21461: Modifica la ley N° 18.101, que fija normas especiales sobre arrendamiento de predios urbanos (Ley Devuélveme mi Casa)*. Recuperado de <https://www.bcn.cl/leychile>
- Biblioteca del Congreso Nacional de Chile. (2024). *Ley 21719: Sobre protección de la vida privada y datos personales (Nueva Ley de Protección de Datos)*. Recuperado de <https://www.bcn.cl/leychile>
- Donovan, A. y Kernighan, B. (2015). *The Go Programming Language*. Addison-Wesley Professional.
- Google Cloud. (2026). *BigQuery documentation: Architecture and Data Warehousing*. Recuperado de <https://cloud.google.com/bigquery/docs>
- Mercado Pago Developers. (2026). *Documentación técnica de Split de Pagos (Marketplace) y retenciones*. Recuperado del portal de desarrolladores de Mercado Pago.